

Branch-Dependent Compression Leakage in DARTS-128: Ternary-Secret Recovery and a Fresh-Message Forgery from 17 Million Signatures

Martin Feussner
Selmer Center, University of Bergen
`martin.feussner@uib.no`

26 September 2026

Provenance and review status. The attack was independently found by an AI system, OpenAI Codex using Daybreak Blue at maximum reasoning effort, without a human first proposing this specific attack. The derivation, implementation, experiments, and initial manuscript were also produced by the AI. At the date above, no human cryptanalyst has independently verified the argument, experiments, or novelty assessment. This is a preliminary disclosure for human review and reproduction.

Abstract

DARTS is a module-lattice signature submitted to the Next-generation Commercial Cryptographic Algorithms Program. Its signer forms $z = y + (-1)^b cs$ with an initially balanced bimodal bit b . Before the main sphere rejection, Algorithm 7 accepts the commitment only if $\text{Comp}(w) = \text{Comp}(w + (1 - b)c\alpha)$. This is a real filter for $b = 0$ and the identity for $b = 1$. A gate-only independence heuristic predicts a branch-zero fraction of 0.44133 among gate-passing attempts for DARTS-128. The fully accepted fraction was 0.44148 in an instrumented experiment with 17 million signatures. The public response z_1 then contains an empirical first-moment signal correlated with the two ternary polynomials (s_0, s_1) .

We give a streaming attack that uses fields decoded from ordinary valid signatures. Negacyclic normal equations estimate all 1,024 coefficients of (s_0, s_1) , and a three-class mixture model gives an initial ternary guess. The remaining ambiguity is completed with the public-key equation using a binary-choice Kannan embedding. Its parity structure makes $\{-1, 1\}$ the shortest possible magnitude for the centered correction coordinates; the decoder explicitly rejects vectors without that shape. Flatter followed by BKZ-20 and BKZ-30 recovers the target vector. We separate the exact score identity from its heuristic scalar-leakage approximation and specify a finite classifier and support search that can be fixed in advance for future tests.

On one deterministic DARTS-128 target, the experiment recovered all 1,536 generated ternary coefficients from 17,000,000 signatures. Collection took 13.14 hours at 359.45 accepted signatures per second on a four-core Intel Core i5-6500 with 16 GiB of RAM. The original completion used retrospectively tuned classifier and support parameters and took 4 minutes 55 seconds with less than 87 MiB of resident memory. A target-secret-independent execution of the finite search on the same transcript recovered the same secret in about 5 minutes. Because that search was itself designed during work on this target, the result does not estimate success over random key generation. The original secret seed K was not recovered; packing the exact ternary secret with a fresh K constructed an equivalent signing

key. It produced a 1,449-byte signature on a message absent from the query set, and the unmodified submitted verifier accepted it. This is a successful fresh-message forgery for one fixed DARTS-128 key. No additional keys or parameter sets have undergone end-to-end recovery tests.

1 Introduction

Fiat–Shamir-with-aborts signatures publish a response obtained by adding a secret-dependent challenge product to a random mask. Rejection sampling must remove the dependence on the signing secret. If one bimodal branch is accepted more often than the other, repeated signatures can retain a small multiple of the long-term key.

DARTS is an NGCC lattice-signature submission based on module-LWE and module-SIS assumptions [1]. It combines bimodal rejection sampling [3] with uniform hyperball sampling and a double-sphere output distribution. The response is compressed to obtain signatures of 1,449 bytes at the 128-bit level. DARTS adds a compression-stability filter before its sphere rejection. That filter is branch dependent: it tests a shifted commitment for one value of the bimodal bit and performs a tautological comparison for the other.

The asymmetry is visible before any ternary-secret-recovery step is applied. Under a uniform-commitment and independent-coordinate model, the gate alone predicts that 44.133 percent of gate-passing samples use branch zero. We observe 44.148 percent among fully accepted signatures. The numerical agreement does not prove that later rejection is branch neutral. In the measured transcript, multiplying the public response by the adjoint of the public challenge convolution and solving normal equations produces scores correlated with both secret polynomials.

The estimator is not accurate enough to round every coefficient directly. At 17 million signatures, a public mixture classifier leaves 47 wrong values among 1,024 coefficients. DARTS’s public key supplies a much stronger final test. For DARTS-128 it satisfies

$$A_0 s_0 + A_1 s_1 + e = h \pmod{q}, \quad s_0, s_1, e \in \{-1, 0, 1\}^{512},$$

where h has constant coefficient $(q + 1)/2$ and all other coefficients zero. Thus a candidate for (s_0, s_1) determines e , and all 512 coefficients of e must be ternary.

Our contributions are:

- We identify the branch-dependent compression gate in the specified DARTS signing algorithm and measure the final branch imbalance on one instrumented target.
- We derive an exact decomposition of a public normal-equation score for the two transmitted secret components and separate that identity from the heuristic scalar-leakage model. The estimator is streaming and stores three ring accumulators.
- We disclose the retrospective choices used in the original 130-variable completion and specify deterministic classifier-width and support ladders with a public validation rule.
- We state the precise decoding condition for the binary-choice Kannan embedding; shortness favors binary coordinates, while an explicit decoder and the complete public relation validate candidates.
- The experiment demonstrates exact ternary-secret recovery, equivalent signing-key construction, and an accepted fresh-message forgery from a 17-million-signature transcript on an ordinary four-core machine.

The attack uses no timing measurement, fault, malformed signature, weak hash output, or exposed signing randomness. The branch-dependent gate appears in the specified procedure and is faithfully implemented by the reference code.

2 Relevant parts of DARTS-128

Let

$$\mathcal{R}_q = \mathbb{Z}_q[X]/(X^{512} + 1), \quad q = 130817.$$

DARTS-128 uses $(k, \ell) = (1, 2)$. Key generation samples three ternary polynomials

$$s = (s_0, s_1, e) \in \mathcal{R}_q^3$$

with independent coefficient probabilities

$$\Pr[s_i = -1] = 0.15, \quad \Pr[s_i = 0] = 0.70, \quad \Pr[s_i = 1] = 0.15.$$

The public matrix is $A = (A_0, A_1, 1)$ and the key relation is

$$A_0 s_0 + A_1 s_1 + e = h \pmod{q}, \tag{1}$$

where $h = ((q+1)/2, 0, \dots, 0)$. The public key determines A_0 and A_1 .

For each signing attempt, the signer samples a hyperball mask y , a bimodal bit b , and an auxiliary rejection bit. It computes a commitment w , then a sparse challenge c of Hamming weight $\tau = 30$. The secret-dependent response is

$$z = y + (-1)^b cs. \tag{2}$$

The signature directly encodes the two-polynomial block z_1 corresponding to (s_0, s_1) , split into compressed high and low parts. Anyone can decode those fields and recover the integer coefficient vector used by verification. The third response component is handled by a hint and is unnecessary for our estimator.

Before computing Equation (2), DARTS Algorithm 7 applies

$$\text{Comp}_d(w) = \text{Comp}_d(w + (1-b)c\alpha), \tag{3}$$

with $d = 10$. When $b = 1$, Equation (3) is $\text{Comp}_d(w) = \text{Comp}_d(w)$ and always passes. When $b = 0$, adding the sparse challenge can cross a compression boundary and force a restart. The later double-sphere rejection is intended to shape z . The final accepted transcript remained imbalanced in the instrumented experiment reported below.

3 From the compression gate to a secret estimator

3.1 Gate-only prediction and fully accepted measurement

For a uniformly distributed coefficient $a \in \mathbb{Z}_q$, the gate model gives

$$\Pr[\text{Comp}_d(a) = \text{Comp}_d(a+1)] \simeq 1 - \frac{2^d}{q}.$$

Treating the τ challenged coefficients as independent gives the heuristic branch-zero gate-pass probability

$$\rho = \left(1 - \frac{2^{10}}{130817}\right)^{30} \simeq 0.78997. \tag{4}$$

The two branches begin with equal probability, so conditioning on this gate model predicts

$$\Pr[b = 0 \mid \text{gate passes}] = \frac{\rho}{1+\rho} \simeq 0.44133, \quad \mathbb{E}[(-1)^b] \simeq -0.11734. \tag{5}$$

This calculation covers the compression gate. It does not model the later sphere rejection, encoding checks, or dependencies among accepted variables. In the 17-million-signature experiment, the hidden branch counter was read only after data collection as a diagnostic. It contained 7,505,081 branch-zero and 9,494,919 branch-one fully accepted signatures. Thus the observed values were

$$\widehat{\Pr}[b = 0] = 0.44147535, \quad \bar{\varepsilon} = -0.11704929,$$

numerically close to Equation (5). This agreement is an empirical observation, not a proof that the later filters preserve the gate-only distribution. The attack itself does not use the hidden branch bit or these counters.

3.2 Public negacyclic normal equations

Let C_t be the 512×512 signed negacyclic convolution matrix of the public challenge c_t in signature t . For response polynomial $p \in \{0, 1\}$, Equation (2) becomes

$$z_{p,t} = y_{p,t} + \varepsilon_t C_t s_p, \quad \varepsilon_t = (-1)^{b_t}. \quad (6)$$

Define the streaming accumulators

$$G = \sum_{t=1}^N C_t^T C_t, \quad r_p = \sum_{t=1}^N C_t^T z_{p,t}. \quad (7)$$

Negacyclic symmetry means G is determined by one 512-coefficient polynomial, so the complete attack state is G, r_0, r_1 plus counters and the public key. No raw signature storage is required. After collection, solve

$$G\theta_p = r_p. \quad (8)$$

For a fixed accepted transcript, define

$$\eta_p = \sum_{t=1}^N C_t^T y_{p,t}, \quad H = \sum_{t=1}^N \varepsilon_t C_t^T C_t.$$

Then the following decomposition is exact:

$$r_p = \eta_p + H s_p, \quad \theta_p = G^{-1} \eta_p + G^{-1} H s_p, \quad (9)$$

whenever G is invertible. This algebraic identity is the rigorous part of the estimator derivation.

The often useful approximation

$$\theta_p \simeq \mathbb{E}[\varepsilon] s_p \quad (10)$$

requires two additional concentration statements: $G^{-1} \eta_p$ must be small, and $G^{-1} H$ must be close to the scalar matrix $\mathbb{E}[\varepsilon] I$. Those statements would follow from suitable independence and zero-correlation hypotheses before conditioning. They are not proved for the distribution after the compression gate, sphere rejection, and encoding checks. Equation (10) is therefore a heuristic explanation, not a theorem about DARTS.

The target measurements also show that the scalar approximation is not quantitatively exact. Regressing the 17-million-signature scores on the known secret only as a post hoc diagnostic gives slopes -0.04160 for s_0 and -0.04159 for s_1 , whereas the measured branch mean is $\bar{\varepsilon} = -0.11705$. The corresponding correlations are -0.8563 and -0.9575 . Thus later conditioning attenuates or otherwise modifies the simple branch mean. The first score also has a stable coordinate-dependent component. These effects prevent direct rounding but leave three empirical class centers.

This estimator uses only c_t and the decoded public $z_{1,t}$. The chosen messages may be arbitrary distinct messages. No internal rejection count, branch bit, mask, or secret coefficient appears in Equations (7)–(8).

3.3 Oracle branch-balancing diagnostic

To test the role of branch proportions, we made one separate diagnostic run of 100,000 fully accepted signatures on the same deterministic key. The run split the public normal equations by the instrumented hidden bit b . An oracle-balanced score then weighted each branch by the reciprocal of its accepted count. This diagnostic uses information unavailable to the attack; it is a causality check rather than an attack step.

The run contained 44,072 branch-zero and 55,928 branch-one signatures. Table 1 reports an ordinary least-squares slope in $\theta = \alpha + \beta s$ and the coefficient correlation, pooling s_0 and s_1 .

Table 1: Same-key 100,000-signature oracle branch-balancing diagnostic.

score construction	slope β	correlation
branch zero only	+0.36275	+0.83206
branch one only	−0.33965	−0.83645
observed mixture	−0.03004	−0.17900
equal branch weight	+0.01160	+0.06989

Equal weighting removes most of the scalar signal and reverses its small remainder in this sample; it does not produce an exact numerical zero. A 100,000-signature diagnostic with dependent ring coefficients is insufficient to distinguish finite-sample fluctuation from a smaller systematic residual. The result supports branch imbalance as a major contributor, while stopping short of claiming that balancing alone proves transcript independence.

4 Public classification and support selection

4.1 Three-class mixture model and retrospective disclosure

The original successful completion used classifier widths and support parameters selected during exploratory work that included comparisons with the target secret. Once those parameters were fixed, the reported completion used only the transcript and public key and applied a public validation rule. We report it for reproducibility, but it is not an out-of-sample evaluation. Sections 4.2 and 4.3 give a fully specified target-secret-independent execution.

The coefficients of each θ_p are classified with the known ternary prior $\pi = (0.15, 0.70, 0.15)$. For class $a \in \{-1, 0, 1\}$, define

$$L_{p,i}(a) = \log \pi_a - \log \sigma_p - \frac{(\theta_{p,i} - \mu_{p,a})^2}{2\sigma_p^2}. \quad (11)$$

The original completion uses $\sigma_0 = 0.0140$ and $\sigma_1 = 0.0083$, with initial ordered means $(0.0376, -0.0041, -0.0442)$. On the target transcript, 100 expectation-maximization iterations update the three means using only θ_p while retaining the public priors and fixed widths. The final guess and confidence are

$$\hat{s}_{p,i} = \arg \max_a L_{p,i}(a), \quad (12)$$

$$\kappa_{p,i} = \max_a L_{p,i}(a) - \text{secondmax}_a L_{p,i}(a). \quad (13)$$

The fitted target means were

polynomial	$a = -1$	$a = 0$	$a = 1$
s_0	0.03589272	−0.00592390	−0.04756463
s_1	0.04202248	−0.00018468	−0.04113097

Post hoc comparison with the target key found that the public guess was correct in 977 of 1,024 positions: 45 errors were in s_0 and two were in s_1 .

4.2 Deterministic public classifier search

A target-secret-independent execution must also fix how the classifier parameters are obtained. The following deterministic procedure uses only the score and the public ternary prior. For each polynomial, sort its 512 scores. Initialize the mean for class -1 from the largest 77 values, the mean for class 0 from the middle 358, and the mean for class $+1$ from the smallest 77. This ordering follows the negative leakage slope. Initialize a common component width with the public sample standard deviation, then run EM with fixed priors while updating both the three means and the shared width:

$$\gamma_{i,a} = \frac{\pi_a \exp(-(\theta_i - \mu_a)^2 / (2\sigma^2))}{\sum_{a'} \pi_{a'} \exp(-(\theta_i - \mu_{a'})^2 / (2\sigma^2))}, \quad (14)$$

$$\mu_a = \frac{\sum_i \gamma_{i,a} \theta_i}{\sum_i \gamma_{i,a}}, \quad \sigma^2 = \frac{1}{512} \sum_{i,a} \gamma_{i,a} (\theta_i - \mu_a)^2. \quad (15)$$

Stop when the largest mean or width change is below 10^{-13} , or after 500 iterations. Denote the resulting widths by $\hat{\sigma}_0, \hat{\sigma}_1$.

To avoid choosing a width after seeing the target secret, enumerate the fixed multiplier set

$$\Lambda = \{1, 9/8, 5/4, 11/8, 3/2\}.$$

For every pair $(\lambda_0, \lambda_1) \in \Lambda^2$, run 100 mean-only EM updates with widths $\lambda_p \hat{\sigma}_p$, then run the public support ladder below. Order the 25 pairs first by $(\max(\lambda_0, \lambda_1), \lambda_0 + \lambda_1, \lambda_0, \lambda_1)$ and break all remaining ties by index. Traverse support rungs 0 through 5 within each width pair. At each trial, apply the fixed reduction schedule in Section 5. Accept only a candidate for which (s_0, s_1) is ternary, the publicly derived e is ternary, all 512 public equations hold, and a fresh signature verifies. Otherwise advance to the next support rung or width pair; if the finite grid is exhausted, report failure at the fixed signature budget.

This defines a proper public attack experiment: fix the signature budget, classifier search, support ladder, reduction schedule, and public stop condition before drawing a target key. The signing-oracle transcript and public key are its only per-target inputs. The procedure can be fixed in advance for future tests. One such evaluation could use a budget of 17 million signatures and record success or failure without reading the generated secret. We do not perform that additional-key experiment here.

4.3 Exploratory support and a deterministic public ladder

Confidence alone misses some high-confidence s_0 errors caused by the fixed component. Their location is also visible publicly as blockwise drift. Partition θ_0 into sixteen consecutive blocks of 32 coefficients. Let m_j be the mean of block j and let m_* be the median of the sixteen block means. Select the three blocks maximizing $|m_j - m_*|$.

At 17 million signatures, this rule ranks blocks 15, 0, and 1 first. Their means were respectively

$$-0.03754359, \quad -0.02454275, \quad -0.01962178, \quad m_* = -0.00582224.$$

The original successful completion took the union of these 96 block positions and the 64 least-confident s_0 positions, leaving 128 positions after overlap. It added the two least-confident s_1 positions and used 48 public equations, for 130 binary choices in a 179-dimensional lattice.

At each selected position, retain the most likely class and its runner-up. Their difference is $a_j \in \{-1, 1\}$. Introduce one bit x_j that chooses between them. Post hoc diagnostics show that

all 47 wrong guesses lie in the support and that the runner-up direction is correct for every one. The final reduction used only public data, and success was detected through Equation (1). These counts are the retrospectively selected parameters disclosed in Section 4.1.

For target-secret-independent execution, use the public ladder in Table 2. At each rung, select the stated number of least-confident positions in each polynomial; unite the s_0 set with every position in the stated number of blocks having largest $|m_j - m_*|$; and keep the public runner-up class. Ties are broken by increasing coefficient or block index; likelihood ties are sorted by increasing class label and the last class is preferred. Construct the embedding with the stated prefix length m and stop only when all 1,536 derived coefficients are ternary and the complete public relation holds.

Table 2: Deterministic public support ladder.

rung	low-confidence s_0	low-confidence s_1	anomalous s_0 blocks	equations m
0	32	2	1	32
1	64	2	2	40
2	64	2	3	48
3	96	8	4	56
4	128	16	6	64
5	160	32	8	72

On the saved transcript, rung 2 publicly selects blocks 15, 0, and 1. Its unions contain 128 positions in s_0 and two in s_1 , recreating the 130-choice, 179-dimensional instance. The reduction schedule in Section 5 returns the same publicly validated ternary secret.

We also executed the deterministic public classifier procedure on the saved transcript. Its fitted base widths were (0.01037953, 0.00715656). Width-pair rung 2, $(\lambda_0, \lambda_1) = (9/8, 1)$, and support rung 2 produced a public guess with 971 of 1,024 coefficients correct in a post hoc comparison. The public unions contained 127 positions in s_0 and two in s_1 , giving a 178-dimensional lattice with 129 choices and 48 equations. Flatter, BKZ-20, and BKZ-30 returned a publicly valid vector selecting all 53 needed corrections in 302.91 seconds of measured reduction time and 89,292 KiB peak resident memory. It again achieved exact ternary-secret recovery. A basis construction without access to the target secret produced a byte-identical raw basis to the diagnostic build.

The ladder and width grid were designed after exploratory work on this target; the implication for the strength of the claim is stated in Section 7.

5 Binary lattice completion

5.1 The modular correction problem

Index the 130 selected corrections by j . Let $D_j \in \mathbb{Z}_q^m$ be the first $m = 48$ coefficients of the public product obtained by placing a_j at its selected secret coordinate and zeros elsewhere. Set

$$D = (D_1 | \cdots | D_u) \in \mathbb{Z}_q^{m \times u}, \quad u = 130, \quad (16)$$

and let

$$r = (h - A_0 \hat{s}_0 - A_1 \hat{s}_1)_{0:m-1} \pmod{q}. \quad (17)$$

The desired correction bits satisfy

$$Dx + e' = r \pmod{q}, \quad x \in \{0, 1\}^u, \quad e' \in \{-1, 0, 1\}^m. \quad (18)$$

Ordinary integer embeddings admit many short vectors whose correction coordinates are small integers but not valid class choices. We instead center each bit as $1 - 2x_j \in \{-1, 1\}$.

5.2 The embedding

Use the following row basis of dimension $d = u + m + 1$:

$$B = \begin{pmatrix} 0_{m \times u} & 2qI_m & 0 \\ 2I_u & 2D^T & 0 \\ \mathbf{1}_u^T & 2r^T & 1 \end{pmatrix}. \quad (19)$$

Lemma 1. *If (x, e') satisfies Equation (18), then the lattice generated by Equation (19) contains*

$$v = (\mathbf{1} - 2x, 2e', 1).$$

Conversely, any lattice vector with last coordinate ± 1 and first u coordinates all in $\{-1, 1\}$ can be oriented to last coordinate 1 and written $(\mathbf{1} - 2x, 2e', 1)$ for some $x \in \{0, 1\}^u$ and $e' \in \mathbb{Z}^m$ satisfying $Dx + e' = r \pmod{q}$. It solves Equation (18) exactly when $e' \in \{-1, 0, 1\}^m$.

Proof. Choose $k \in \mathbb{Z}^m$ such that $r - Dx + qk = e'$. Add the final row of B , subtract x_j times binary row j , and add the corresponding modulus rows. The three blocks are $\mathbf{1} - 2x$, $2e'$, and 1. For the converse, orient the vector so its last entry is 1. Its integer combination contains the final row once. The first block is therefore $\mathbf{1} + 2t$ for an integer vector t ; the assumed $\{-1, 1\}$ shape gives $x = (\mathbf{1} - v_{0:u})/2 = -t \in \{0, 1\}^u$. Dividing the middle block by two defines an integer e' and gives $Dx + e' = r \pmod{q}$. The ternary bound on e' , the remaining 464 public equations, and the bounds on the complete derived ternary vector reject false candidates. $\square$

The embedding does not force its first coordinates to be binary. With last coordinate oriented to 1, it forces only odd parity, so coordinates such as ± 3 are also possible. Euclidean shortness favors the minimum absolute value 1, and the decoder explicitly accepts only the $\{-1, 1\}$ shape. The lemma is an algebraic encoding and decoding statement; it is not a uniqueness or shortest-vector theorem. Recovery by the stated reduction schedule is an experimental fact for the demonstrated instance.

On the demonstrated target, 12 of the first 48 coefficients of e are nonzero. Hence

$$\|v\|^2 = 130 + 4 \cdot 12 + 1 = 179. \quad (20)$$

The lattice also has dimension 179 and determinant

$$\det(B) = 2^{130}(2q)^{48}.$$

Its Gaussian-heuristic length is approximately 151.92, whereas the target length is $\sqrt{179} \simeq 13.38$, a ratio of 11.35. This large separation makes modest lattice reduction sufficient despite the structured basis.

5.3 Reduction, validation, and signing

The basis was first reduced with the iterated-compression method of Ryan and Heninger [5], targeting root-Hermite factor 1.01. We then applied two loops of BKZ-20 followed by two loops of BKZ-30 with 256-bit MPFR orthogonalization using fpylll 0.6.4 [6, 7]. The first row after BKZ-30 had the required binary shape. It selected 47 corrections.

The corrected (s_0, s_1) determines

$$e = h - A_0 s_0 - A_1 s_1 \pmod{q}, \quad (21)$$

with centered representatives. Every coefficient of s_0, s_1, e was ternary, so all 512 public equations validated. Post hoc comparison showed that the recovered 1,536 ternary coefficients equal those generated in the original target key exactly.

DARTS stores the public key, (s_0, s_1, e) , and a secret seed K in its signing key. The experiment does not recover the original K . Once the ternary relation is recovered, the attacker can choose a fresh K and pack an equivalent signing key. The new seed changes only the deterministic masks; it does not change Equation (1). We therefore describe the result precisely as *exact ternary-secret recovery followed by equivalent signing-key construction*. The ordinary signing algorithm can then sign fresh messages under the victim public key.

6 End-to-end experiment

6.1 Setup and controls

The experiment used the submitted DARTS archive with SHA-256 digest

1846cfe63f0cef83e2e0ca21f5dcadce3c4b16da33713957be6d156e2a9e6e95.

The machine had a four-core Intel Core i5-6500 at 3.20 GHz and 16 GiB of RAM, running 64-bit Linux. Data collection used eight OpenMP workers. The collector cached only key expansion and NTT values that are invariant across messages; all sampling, hashing, rejection, rounding, compression, and encoding steps were unchanged.

Three controls checked that this optimization did not create the leakage:

- Across four keys and 128 messages per key, cached and submitted signers produced byte-identical signatures, identical accepted branch bits, and identical rejection counts for all 512 cases. Every signature verified.
- Serial and four-thread collection produced byte-identical results and rejection counts for 1,000 messages, with zero mismatches.
- A separate four-thread run generated and verified 10,000 signatures, with zero failures.

The target key was generated deterministically for reproducibility. Query messages were distinct 16-byte counter encodings. The final forgery message was a different ASCII string and did not occur in the query set.

6.2 Collection and statistical recovery

Table 3 reports the collection result. The wall time sums three resumable segments. Hidden branch counts and target-key comparisons are diagnostic values read only after the public attack state was complete.

Table 3: DARTS-128 transcript collection on one target key.

quantity	result
accepted signatures	17,000,000
signing attempts	82,136,603
attempts per accepted signature	4.83156
branch-zero / branch-one (diagnostic)	7,505,081 / 9,494,919
total collection wall time	47,294.659 s (13.137 h)
accepted-signature rate	359.45 s ⁻¹
collector peak resident memory	3.3 MiB
public initial guess	977 / 1,024 correct
errors in s_0 / s_1 (diagnostic)	45 / 2

Storing all fixed-size signatures would require about 24.6 GB. The streaming normal equations instead require only three 512-entry signed accumulators and the public key. The resulting public score state, excluding the integer accumulators used for resumption, is 9,336 bytes.

6.3 Completion and forgery

Table 4 gives the original fixed-parameter completion timings. The signature collector was still occupying the same four-core machine, so wall time includes substantial CPU contention. Once its parameters were fixed, the lattice run received no target secret, branch bits, or hidden rejection data. Section 4.1 discloses how those parameters were selected.

Table 4: Original fixed-parameter completion timings.

stage	wall time	peak resident memory
basis construction and public support	< 1 s	80 MiB
iterated-compression reduction	41.80 s	80.9 MiB
BKZ-20, two loops	82.46 s	81.7 MiB
BKZ-30, two loops	169.71 s	86.1 MiB
streamlined completion total	approximately 295 s	86.1 MiB
fresh signing and verification	0.01 s	1.8 MiB

BKZ-30 returned a valid binary vector as its first basis row. It changed exactly 47 classified coefficients. Equation (21) produced a fully ternary error polynomial and the complete public relation held. After the run, comparison with the known test key gave 1,024 of 1,024 correct coefficients in (s_0, s_1) and 512 of 512 in e .

We packed the recovered ternary secret with an independently chosen 32-byte signing seed and signed the fresh message

fresh DARTS message signed with the lattice-recovered equivalent key.

The resulting signature had the specified length of 1,449 bytes. The unmodified submitted DARTS-128 verifier returned acceptance. Thus the output is a fresh-message forgery, rather than only a distinguisher or partial-key leak.

7 Scope, limitations, and mitigation

The experiment establishes exact ternary-secret recovery, equivalent signing- key construction, and a fresh-message forgery for one fixed DARTS-128 key. In the formal EUF-CMA experiment, the adversary’s success probability includes the randomness of KeyGen. One successful key, particularly one used during parameter exploration, does not estimate that probability. The justified claim is therefore *a successful fresh-message forgery for one fixed DARTS-128 key*. Calling DARTS-128 practically broken in general would require evidence that the method succeeds with non-negligible probability over fresh keys. Because the original seed K was not recovered, we use the terms *exact ternary-secret recovery* and *equivalent signing-key construction* throughout.

Shorter profiling experiments on independently generated keys showed similar three-class first-moment structure, but no additional end-to-end recovery was attempted. The mixture widths, original support parameters, public ladder, and 17-million query count are empirical and were not fixed before the reported experiment. Their portability to fresh random keys is unmeasured.

We make no experimental claim for DARTS-256 or DARTS-512. Their dimensions, challenge weights, and compression parameters differ. The same structural question applies because Algorithm 7 uses the branch-dependent factor $(1 - b)$ at every level, but practical extrapolation requires separate data.

The attack is not an implementation-only side channel. Equation (3) appears in the normative signing algorithm, and the reference implementation faithfully performs it. Restricting timing information or rewriting the code in constant time does not remove the public transcript bias.

A candidate repair should make the commitment filter independent of the bimodal branch, or use a symmetric condition whose accepted response distribution has a proof covering the complete compressed transcript. The oracle diagnostic shows that equal branch weighting removes most of the measured scalar signal, but it does not prove that balancing alone eliminates all public dependence. Any revision should be tested for public correlations $\sum C_t^T z_t$ at large query counts and supported by a new distribution argument.

Seventeen million chosen-message queries are more than typical protocol use, but they were practical for this instance: the complete transcript was generated in about 13 hours on a decade-old four-core processor. Query limits can reduce exposure, while a cryptographic repair requires addressing the accepted-transcript dependence itself.

8 Public record and novelty check

As of 26 September 2026, the public DARTS page at ngcc.dev listed one finding: a 256-bit collision ceiling for the DARTS-512 message representative [8]. It did not describe branch-dependent leakage, exact ternary-secret recovery, or a DARTS-128 forgery. We also searched the public NGCC coverage, the NIST pqc-forum index, and the Cryptology ePrint index. The contemporary overview of early NGCC findings likewise reports no DARTS ternary-secret-recovery attack [9]. We found no prior public account of this attack. This novelty check is necessarily preliminary and awaits human review.

9 Conclusion

DARTS’s compression-stability gate treats its two bimodal branches differently, and the fully accepted transcript was branch imbalanced in the reported experiment. Public responses from 17 million signatures yielded scores correlated with the two ternary secret polynomials; a binary-choice lattice embedding then used the public relation to repair the remaining errors. On the demonstrated key, collection took about 13.14 hours and the original completion took about five minutes on an ordinary four-core machine. The experiment recovered the exact ternary secret, constructed an equivalent signing key with a fresh seed, and produced an accepted signature on a fresh message.

References

- [1] Y. Pan, Q. Lu, Y. Wu, J. Li, C. Cheng, R. Liu, and Q. Du. *DARTS: A Lattice-Based Digital Signature Scheme – Algorithm Specification and Supporting Documentation*. NGCC submission, 29 June 2026.
- [2] V. Lyubashevsky. Fiat–Shamir with aborts: Applications to lattice and factoring-based signatures. In *ASIACRYPT 2009*, pages 598–616, 2009.
- [3] L. Ducas, A. Durmus, T. Lepoint, and V. Lyubashevsky. Lattice signatures and bimodal Gaussians. In *CRYPTO 2013*, pages 40–56, 2013.
- [4] J. Devevey, O. Fawzi, A. Passelegue, and D. Stehle. On rejection sampling in Lyubashevsky’s signature scheme. In *ASIACRYPT 2022*, pages 34–64, 2022.
- [5] K. Ryan and N. Heninger. Fast practical lattice reduction through iterated compression. In *CRYPTO 2023*, pages 3–36, 2023.

- [6] C.-P. Schnorr and M. Euchner. Lattice basis reduction: Improved practical algorithms and solving subset sum problems. *Mathematical Programming*, 66(1):181–199, 1994.
- [7] The FPLLL development team. *fpylld*, a *Python* wrapper for the *fplld* lattice reduction library, version 0.6.4, 2025. <https://github.com/fplld/fpylld>.
- [8] ngcc.dev. DARTS (sign-08) public findings. <https://ngcc.dev/reports/sign-08.html>, accessed 26 September 2026.
- [9] M. Ivezic. China’s Next-Generation Crypto Candidates Drew 104 Public Findings in Three Days. <https://postquantum.com/security-pqc/china-ngcc-candidate-flaws/>, 23 September 2026.